

Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

load Dataset

```
boston_dataset = fetch_openml(name='boston', version=1)
```

```
df= boston_dataset.frame
```

df

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273.0	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273.0	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273.0	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273.0	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273.0	21.0	396.90	7.88	11.9

506 rows × 14 columns

df.describe()

	CRIM	ZN	INDUS	NOX	RM	AGE	DIS	TAX	PTRATIO	B
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.554695	6.284634	68.574901	3.795043	408.237154	18.455534	356.674032
std	8.601545	23.322453	6.860353	0.115878	0.702617	28.148861	2.105710	168.537116	2.164946	91.294864
min	0.006320	0.000000	0.460000	0.385000	3.561000	2.900000	1.129600	187.000000	12.600000	0.320000
25%	0.082045	0.000000	5.190000	0.449000	5.885500	45.025000	2.100175	279.000000	17.400000	375.377500
50%	0.256510	0.000000	9.690000	0.538000	6.208500	77.500000	3.207450	330.000000	19.050000	391.440000
75%	3.677083	12.500000	18.100000	0.624000	6.623500	94.075000	5.188425	666.000000	20.200000	396.225000
max	88.976200	100.000000	27.740000	0.871000	8.780000	100.000000	12.126500	711.000000	22.000000	396.900000

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    CRIM        506 non-null    float64
1    ZN          506 non-null    float64
2    INDUS       506 non-null    float64
3    CHAS        506 non-null    category
```

```

4 NOX      506 non-null float64
5 RM       506 non-null float64
6 AGE      506 non-null float64
7 DIS      506 non-null float64
8 RAD       506 non-null category
9 TAX       506 non-null float64
10 PTRATIO  506 non-null float64
11 B        506 non-null float64
12 LSTAT    506 non-null float64
13 MEDV     506 non-null float64
dtypes: category(2), float64(12)
memory usage: 49.0 KB

```

Start coding or [generate](#) with AI.

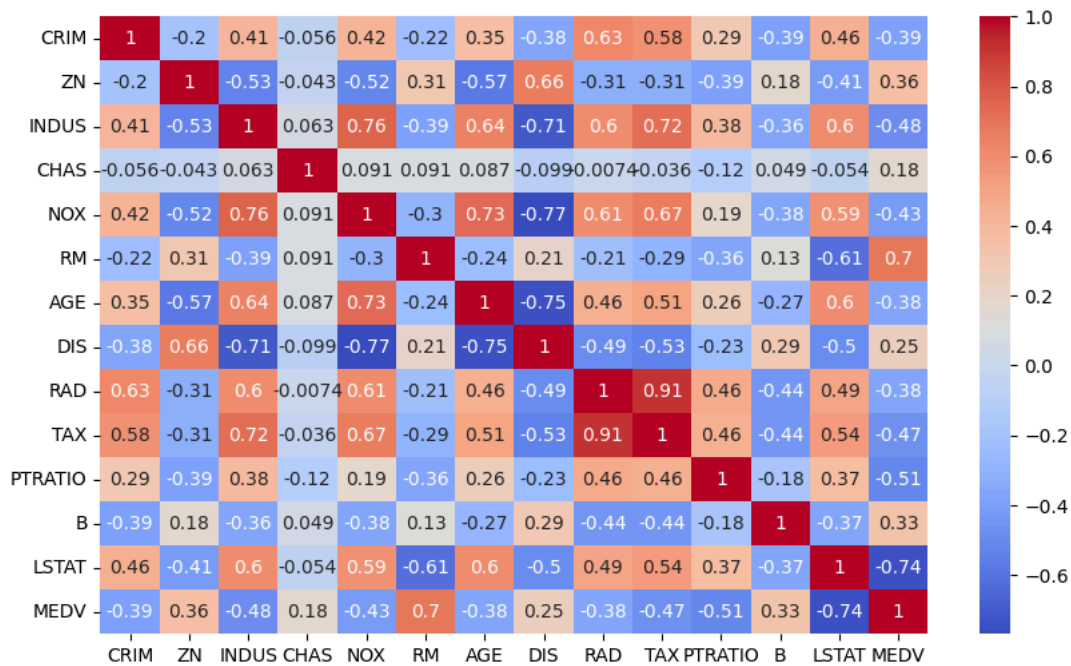
▼ Data Exploration

```

plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt

```

<module 'matplotlib.pyplot' from '/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py'>



▼ Feature and Target

```

X=df.drop('MEDV',axis=1)
Y=df['MEDV']

```

df.head()

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

Y.head()

```
MEDV
0    24.0
1    21.6
2    34.7
3    33.4
4    36.2

dtype: float64
```

Train Test Split

```
X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size=0.2,random_state=42)
```

```
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
```

```
X_train
array([[ 1.28770177, -0.50032012,  1.03323679, ...,  0.84534281,
        -0.07433689,  1.75350503],
       [-0.33638447, -0.50032012, -0.41315956, ...,  1.20474139,
         0.4301838 , -0.5614742 ],
       [-0.40325332,  1.01327135, -0.71521823, ..., -0.63717631,
         0.06529747, -0.65159505],
       ...,
       [-0.40547014,  2.95931752, -1.30336132, ..., -0.59225149,
         0.37901005, -0.91069248],
       [ 0.85189733, -0.50032012,  1.03323679, ...,  0.84534281,
        -2.69458597,  1.52257036],
       [-0.38135592, -0.50032012, -0.35216694, ...,  1.15981657,
        -3.12158061, -0.25731635]])
```

Train

```
model=LinearRegression()
model.fit(X_train,y_train)
```

```
LinearRegression
```

Start coding or [generate](#) with AI.

slope and Intercept

```
coefficient=model.coef_
intercept=model.intercept_
feature_names=X.columns

coef_df =pd.DataFrame({'Feature':feature_names,'Coefficient':coefficient})
print(coef_df)
```

```
Feature  Coefficient
0      CRIM    -1.002135
1       ZN      0.696269
2     INDUS    0.278065
3      CHAS    0.718738
4      NOX    -2.022319
5       RM     3.145240
6      AGE    -0.176048
7      DIS    -3.081908
8      RAD     2.251407
9      TAX    -1.767014
10  PTRATIO    -2.037752
11       B     1.129568
12    LSTAT    -3.611658
```

```
intercept
```

```
np.float64(22.796534653465343)
```

```
Y_pred = model.predict(X_test)
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)
```

```
print(f'Mean Squared Error: {mse:.2f}')
print(f'R-squared: {r2:.2f}')
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_8672/164469831.py in <cell line: 0>()
----> 1 Y_pred = linear_model.predict(X_test_scaled)
      2 Y_pred = model.predict(X_test)
      3 mse = mean_squared_error(Y_test, Y_pred)
      4 r2 = r2_score(Y_test, Y_pred)
      5

NameError: name 'linear_model' is not defined
```

Start coding or [generate](#) with AI.